From Visual Ontologies to Executable Representations: A Framework for AI-Assisted Software Architecture

The Semantic Chasm — From Visual Intent to Computational Ambiguity

The practice of software architecture is fundamentally an act of communication. To manage the immense complexity of modern software systems, architects and engineers rely on visual languages—diagrams of boxes, lines, and icons—to convey structure, relationships, and intent. These diagrams are indispensable tools, serving as the blueprints that guide development, facilitate stakeholder communication, and accelerate the onboarding of new team members.[1] They allow teams to simplify complexity, ensure design consistency, and make informed decisions about the evolution of a system.[1] However, this expressive power comes at a cost. The very informality that makes these diagrams accessible to a wide range of stakeholders, from business managers to operators, also introduces a profound and dangerous ambiguity.[3] This initial section establishes this core problem domain: the chasm that separates high-level, human-centric visual artifacts from the low-level, formally specified logic required for machine interpretation and automated code generation. It explores the dual nature of diagrams as both powerful communication tools and deep wells of computational ambiguity, setting the stage for a framework designed to bridge this gap.

The Expressive Power and Inherent Limitations of Visual Languages

Software architecture diagrams, whether they adhere to a formal standard like the Unified Modeling Language (UML) or consist of ad-hoc "boxes and lines," function as a critical abstraction layer.[3] They allow for the analysis of a system's behavior and quality attributes before a single line of code is written, providing a basis for

cost-saving, risk mitigation, and the reuse of established patterns and decisions.[3] This visual representation is a language, one that uses spatial arrangement, symbols, and connections to describe a system's components and their interactions.[5]

However, the meaning of this language is often implicit and context-dependent. Most architectural descriptions are informal, relying on conventionalized interpretations to be understood.[4] A box might represent a microservice in one diagram, a database in another, and an abstract concept in a third. A line might signify a synchronous API call, an asynchronous message queue, or a simple data dependency. This imprecision has severe limitations. Without a rigid set of rules to define their meaning, diagrams become ambiguous and can lead to misunderstandings.[7] A computer, unlike a human engineer, cannot act on assumptions or guess the intended meaning of a symbol based on its visual context; it requires a complete and unambiguous description of both the syntax (the allowable shapes and connections) and the semantics (the meaning of those shapes and connections).[8]

This fundamental challenge can be productively reframed by classifying visual architecture diagrams as a form of *informal Domain-Specific Language (DSL)*. A DSL is a computer language specialized for a particular application domain, designed to solve problems within that domain with greater expressive power than a general-purpose language.[9] Architecture diagrams fit this definition perfectly: they employ a specific vocabulary (a visual syntax of nodes, connectors, and icons) to solve a narrow set of problems (describing software structure, behavior, and deployment).[9] The "informal" qualifier is crucial; it acknowledges that while these diagrams possess a syntax and are domain-specific, their semantics are not formally defined and rely heavily on human interpretation.

Viewing diagrams as informal DSLs is not merely an academic exercise. It transforms the problem from the vague task of "understanding a picture" into the well-understood computer science problem of *language processing*. The challenge becomes one of parsing an input string (the visual diagram), analyzing its structure, and compiling it into a formal, machine-executable representation. This perspective provides a clear path forward: to bridge the semantic chasm, one must build a compiler for the informal visual language of software architecture. This compiler's target language will not be machine code, but a structured, machine-interpretable specification that an AI agent can execute.

Semantic Architecture: Formalizing the Meaning of "Boxes and Lines"

The inherent ambiguity of informal diagrams has long been a recognized problem in software engineering. The proposed solution, emerging from the field of software architecture research, is the concept of **Semantic Architecture**. This approach advocates for the creation of architecture descriptions that are formal, machine-readable, and capable of unambiguously capturing, cataloging, and exchanging semantic information.[11] The primary goals of semantic architecture are to support the automatic generation of software models from high-level descriptions and to permit the rigorous analysis of architectural quality attributes.[11]

To achieve this, semantic architecture proposes the creation of a "semantic layer" that sits between the complex, heterogeneous data sources of an enterprise and the front-end applications that consume them.[12] This layer provides a business-centric, "whiteboardable" view of enterprise data and its relationships, making it accessible to both technical and non-technical users. It does this by using formalisms like ontologies and knowledge graphs to map simple business terms and their context to the complex underlying data structures. In essence, it provides a formal model for the meaning of the "boxes and lines" used in informal diagrams.

The user query at the heart of this report can be understood as a request for a practical, end-to-end implementation of the principles of Semantic Architecture. The prompt's requirements—to dissect visual nodes as carriers of ontological identity, to translate their meaning into a machine-interpretable specification like JSON-LD, and to synthesize this into an executable bundle for an AI—represent a direct operationalization of the theoretical goals of semantic architecture. The entire process described, from visual deconstruction to the generation of an Executable Context Bundle (CxB), constitutes a concrete blueprint for building the very system that semantic architecture theory has long envisioned. This report, therefore, is not a theoretical exploration but a practical guide to achieving this long-standing objective in software engineering, leveraging modern AI capabilities as the enabling technology.

The Problem of Symbolic Drift and Contextual Decay

Even when an initial mapping from a diagram to an implementation is achieved, a persistent and pernicious problem remains: **symbolic drift**. Over time, as a software system evolves through maintenance, refactoring, and feature additions, the

implementation inevitably diverges from the original architectural diagrams. The meaning of the symbols in the diagram "drifts" away from the reality of the code. A box labeled "User Service" might have its responsibilities split into three new microservices, but the diagram is never updated to reflect this change. This decay of fidelity between the visual representation and the computational reality is a primary reason why architectural documentation loses its value and becomes a source of technical debt rather than a guide.

This phenomenon can be understood through the lens of visual semantics, which studies the interpretation of visual elements.[13] Every symbol in a diagram has a

denotation (its literal meaning, e.g., a box is a component) and a *connotation* (the cultural or emotional associations, e.g., a box with a specific color might imply a certain technology stack or criticality level). Symbolic drift occurs when the denotation remains fixed (the box is still a box) but its underlying meaning in the implemented system has changed, creating a cognitive disconnect for the human reader and a fatal error for a machine interpreter.

Within the framework of Context Engineering, this problem can be diagnosed more precisely. The literature on AI agents identifies "context failure"—the lack of necessary information at runtime—as a primary cause of poor performance.[14] An agent that lacks chat history or is given an irrelevant document via Retrieval-Augmented Generation (RAG) will likely fail its task. Symbolic drift in an architectural diagram represents a more fundamental type of failure: a

design-time context failure. The architectural blueprint, which should serve as the foundational context for an AI coding assistant, is itself flawed, outdated, or misleading. When an AI agent bases its reasoning on a diagram suffering from symbolic drift, it is operating from a position of profound ignorance, no matter how sophisticated its reasoning capabilities are.

This realization carries a critical implication for any system designed to translate visual semantics into executable logic. The translation cannot be a one-time, unidirectional process. A robust system must treat the architecture as a *living artifact*. It requires mechanisms for **bi-directional synchronization**, where changes in the formal model (the machine-interpretable specification) can be reflected back into the visual diagram, and where deviations in the code from the established architecture can be flagged. Without this continuous feedback loop, any attempt to formalize visual semantics is doomed to obsolescence, and the semantic chasm will inevitably

reappear.

Context Engineering as the Bridge — A New Discipline for AI Systems

To construct a durable bridge across the semantic chasm, a new engineering discipline is required—one that moves beyond the tactical crafting of individual prompts and embraces the architectural challenge of providing AI models with a complete, dynamic, and reliable understanding of their operational environment. This discipline is **Context Engineering**. It represents a paradigm shift from treating Large Language Models (LLMs) as simple input-output functions to architecting the entire information ecosystem in which they operate. This section will define Context Engineering, differentiate it from its predecessor, prompt engineering, and establish it as the core methodology that makes the translation from informal visual semantics to formal executable logic not just possible, but robust and scalable.

From Prompt Crafting to Context Architecture

The initial wave of interaction with LLMs was dominated by **prompt engineering**, the art and science of carefully crafting a single text string to elicit a desired response.[15] While a crucial skill, prompt engineering is fundamentally a "local optimization".[16] It focuses on perfecting one turn in a conversation, one query to the model.

Context Engineering, by contrast, is a strategic, system-level discipline. It is the practice of designing and building dynamic systems that provide an LLM with the right information and the right tools, in the right format, at the right time, to successfully accomplish a complex task.[14]

The fundamental shift is from viewing the input as a static string to architecting the *context window* as a managed resource.[18] This context is not just the user's immediate prompt; it is the dynamic output of a pre-processing system that runs before the main LLM is ever called.[14] This distinction is profound and has far-reaching architectural implications.

A helpful analogy is to consider the relationship between a traditional computer program and its operating system. A program does not run in a vacuum; it executes within a runtime environment that provides managed access to memory (RAM), persistent storage (filesystems), and external capabilities (system calls). The operating system is responsible for loading the right data into memory, managing limited resources, and providing a stable interface for the program to interact with the world.

From this perspective, Context Engineering is the discipline of building the **runtime environment for an agentic LLM**. The context engineering system serves as the AI's operating system. It manages the LLM's "working memory" (the limited context window), deciding what information is critical and what is noise, much like an OS swaps pages in and out of RAM.[18] It provides access to "filesystems" in the form of knowledge bases, which are queried via Retrieval-Augmented Generation (RAG). And it furnishes the LLM with "system calls" in the form of tools and APIs that allow it to take action in the world.[14] This reframing elevates Context Engineering from a mere "advanced prompting technique" to a fundamental architectural practice, as essential to building reliable AI agents as an operating system is to running reliable software.

Feature	Prompt Engineering	Context Engineering
Scope	Single interaction, single turn	System-level, multi-turn task
Nature	Static string, template-based	Dynamic output of a pre-processing system
Goal	Elicit a specific response	Enable the successful completion of a task
Analogy	Crafting a perfect database query	Architecting the application's runtime environment
Failure Mode	A bad or unhelpful response	A "context failure" leading to agent malfunction
Key Artifact	A prompt template	A context bundle (e.g., PRP, CxB)

The Components of a Rich Context

To enable an AI agent to perform non-trivial tasks, its runtime environment must assemble a rich, multi-faceted context. This goes far beyond a simple instruction. A comprehensive context, as described in the literature, is a composite of several distinct layers of information, dynamically tailored to the specific task at hand.[17] The key components include [14]:

- **System Prompt / Instructions:** The foundational set of rules, roles, and guidelines that govern the AI's behavior throughout an entire session.
- **User Input / Prompt:** The user's immediate request or question.
- **Short-Term Memory / Chat History:** The record of the current conversation, providing immediate conversational context.
- **Retrieved Information (RAG):** External knowledge retrieved from documents, databases, or APIs to ground the AI's response in factual, up-to-date information.
- **Available Tools:** A schema or definition of the functions, APIs, or other capabilities the AI can invoke to take action or gather more information.
- **Structured Output Definitions:** A schema (e.g., JSON Schema) that specifies the required format for the AI's response, ensuring machine-readability.

Assembling these components effectively is what separates a "cheap demo" from a "magical product" that feels genuinely intelligent and helpful.[14] However, simply cramming all available information into the context window is a recipe for failure due to its inherent size limitations. This is where

Workflow Engineering becomes a critical, related discipline.[18]

Workflow engineering takes a step back from the individual LLM call to ask: what is the optimal *sequence* of LLM and non-LLM steps required to reliably complete this work?.[18] Instead of a single, monolithic prompt that asks the AI to solve a complex problem in one shot, a workflow breaks the problem down into a series of focused, manageable steps. Each step in the workflow can have its own, precisely engineered context, thus preventing context overload and improving reliability.[18]

This leads to a powerful synthesis between these two concepts. If workflow engineering defines the *sequence* of steps, and context engineering defines the *information* for each step, then what is the artifact that captures this for the AI? The **Product-Requirements Prompt (PRP)** provides the answer. A PRP, with its layered structure encompassing Business Context, Stakeholder Analysis, Functional

Requirements, and Technical Specifications, serves as a comprehensive snapshot of the total context required for a single, complex workflow.[21] It is the declarative, serialized representation of that workflow's context. The AI agent's job is to "execute" this workflow, using the PRP not as a simple prompt, but as its master specification document. This provides a tangible link between the abstract idea of an "agentic workflow" and the concrete, engineerable artifact of the PRP.

The PRP Architecture — A Formal Grammar for Functional Specification

To bridge the gap between human intent and machine execution, a formal language is needed—one that can capture the rich semantics of a software component's purpose, inputs, outputs, and constraints in a way that is both human-readable and machine-interpretable. The **Product-Requirements Prompt (PRP)**, a core concept in Context Engineering 2.0, serves as this formal language. This section provides a deep analysis of the PRP architecture, dissecting its structure and the "Context-as-a-Compiler" workflow it enables. It positions the PRP not merely as a structured prompt, but as a formal grammar for defining software functionality for an AI agent.

Anatomy of a Product-Requirements Prompt (PRP)

A Product-Requirements Prompt (PRP) is a structured artifact designed to bridge the chasm between unstructured business conversations and the actionable, technical specifications required for AI-driven development.[21] It transforms scattered ideas and stakeholder requests into a coherent specification by applying a layered context architecture. This systematic approach ensures that critical information is not lost in translation, providing the AI with a holistic understanding of the task.[21]

The concrete JSON schema provided in the user's query serves as an excellent example of a PRP tailored for defining a single function. Its structure includes key fields that systematically capture both functional and non-functional requirements:

- **function_name**: The unique identifier for the computational unit.

- **description**: A natural language explanation of the function's purpose.
- **role**: An ontological classification of the function's purpose within the broader system (e.g., intent_interpreter, data_transformer, api_gateway).
- **input_spec**: Defines the data and conditions required to invoke the function.
- **output_spec**: Defines the data structure and state guaranteed upon successful completion.
- **constraints**: Specifies rules, limitations, and fallback behaviors that govern the function's execution.
- **purpose_vector**: A list of tags that align the function with higher-level strategic goals (e.g., semantic_grounding, zero-shot_usability).
- **meta**: Administrative data such as version, author, and epistemic status (e.g., stable, inferred).

This structure is more than just a well-organized data format. It constitutes a **declarative Domain-Specific Language (DSL) for defining function contracts**. A DSL is characterized by a specialized syntax and semantics tailored to a particular problem domain.[9] The PRP schema fits this definition perfectly:

1. **Specialized Syntax:** The PRP uses a rigid JSON structure with predefined keys and expected value types. This is its grammar.
2. **Specific Domain:** Its domain is the unambiguous specification of software functions for consumption by an AI agent.
3. **Formal Semantics:** The meaning of each field is precisely defined by how the AI agent is programmed to interpret it. For example, the agent understands that the contents of input_spec map to preconditions that must be satisfied before execution, and the structure in output_spec defines the postconditions it must guarantee.

Recognizing the PRP as a formal DSL has significant practical implications. It means that we can apply the principles and tools of language engineering to it. We can build linters to validate the syntactic correctness of a PRP file. We can create static analyzers to check for logical inconsistencies (e.g., a dependency on a non-existent function). Most powerfully, we can develop compilers that transform PRPs from this declarative format into other useful artifacts, such as human-readable documentation, automated test harnesses, or monitoring configurations. The PRP, therefore, becomes the single source of truth for a function's specification, from which other representations are derived.

The "Context-as-a-Compiler" Workflow

The PRP is not a standalone artifact; it is the centerpiece of a dynamic, multi-stage workflow designed for AI-driven software development. This process, termed the "Context-as-a-Compiler" workflow, provides a powerful mental model: just as a traditional compiler requires all source files, headers, and library dependencies to produce a correct executable, an LLM requires a complete and well-structured context to generate correct and reliable code.[16] A missing piece of context is not just an inconvenience; it is a "compilation error" that leads to a "runtime error" in the form of a hallucination or incorrect behavior.[16]

This workflow consists of four distinct stages [16]:

1. **Set Up Global Rules (CLAUDE.md):** This initial step involves creating a project-wide configuration file. This file acts like a set of global compiler flags or project-wide headers. It defines the "rules of the road" for the AI assistant, specifying coding standards, preferred testing frameworks (e.g., "use Pytest with fixtures"), documentation formats, and architectural principles. This ensures that all AI-generated code is consistent and adheres to the project's established conventions.
2. **Create Initial Feature Request (INITIAL.md):** This document is the "source code" for the new feature or component. It is a highly structured, yet human-authored, file that provides the AI with the high-level intent. It contains explicit sections for a detailed FEATURE description, EXAMPLES of existing code patterns to emulate, links to relevant DOCUMENTATION, and a section for OTHER CONSIDERATIONS to capture non-obvious constraints or potential edge cases.
3. **Generate the PRP (/generate-prp):** This is a critical, agentic step that functions as the "pre-compilation" phase. Here, the AI assistant takes the INITIAL.md file as input and performs autonomous research. It analyzes the existing codebase to identify relevant patterns, fetches and reads the specified documentation, and synthesizes all of this information into a comprehensive implementation blueprint—the PRP. This generated PRP is far more detailed than the initial request. It includes a step-by-step execution plan, specifies error handling patterns, and crucially, defines "validation gates" (e.g., specific test commands that must pass) for each step of the plan.
4. **Execute the PRP (/execute-prp):** This is the final "compile and test" phase. The AI agent loads the entire context from the generated PRP and begins to execute the plan step-by-step. After generating the code for each step, it immediately runs the corresponding validation gate. If a test fails, the system enters a tight,

iterative debug loop where the AI attempts to diagnose the error, fix the code, and re-run the test until it passes. This closed-loop, test-driven process ensures the final output is not merely generated, but is validated, functional, and correct with respect to its specification.

A deeper examination of the /generate-prp step reveals its significance. This is not a simple translation or reformatting task. It is an act of **AI-driven requirements synthesis**. In traditional software development, a human systems analyst is responsible for taking high-level business requirements and, through research and analysis, synthesizing a detailed technical design specification. The /generate-prp step automates this very process. The AI agent acts as the systems analyst, taking a high-level intent (INITIAL.md) and autonomously performing the necessary research and planning to produce a detailed, executable specification (the PRP). This represents a qualitative leap in AI-assisted development, moving the AI from a simple "text-to-code" tool to an active partner in the earliest stages of the design and planning lifecycle.

Deconstructing the Diagram — A Systematic Translation Protocol

The theoretical foundations of Semantic Architecture and Context Engineering provide the "why" and "what" of translating visual intent into executable logic. This section provides the "how": a practical, systematic protocol for deconstructing a visual software architecture diagram and mapping its elements into the formal PRP schema. This process is methodical, beginning with a high-level parsing of the diagram's structure and progressively enriching this structure with formal semantics derived from the principles of Design by Contract.

A Protocol for Visual Deconstruction

The translation process begins with a systematic parsing of the visual artifact. This is not a single-pass operation but an iterative process of identification, classification, and interpretation. The goal is to move from a collection of shapes and lines to a structured graph of components and relationships, ready for formalization. The

methodology must be systemic, accounting not only for the components shown but also for the broader contextual environment in which the software will operate.[22]

A robust protocol for this deconstruction follows these steps:

1. **Identify System Boundaries and Actors:** The first step is to establish the scope of the system being modeled. This corresponds to creating a **System Context** diagram in the C4 model.[1] All entities outside the main system boundary are identified as
   Actors (e.g., human users, external software systems). These actors are the primary sources of input and triggers for the system's behavior. In the PRP, an actor like a "User" will map to the source field in an input_spec, often with an input_type of natural_language.[2]
2. **Enumerate Core Components (Nodes):** Next, every primary shape within the system boundary (typically a box or a labeled icon) is enumerated. Each of these **Nodes** represents a major computational component or data store. For each node, a conceptual role must be assigned (e.g., API Gateway, Authentication Service, Product Database). This initial identification and classification directly informs the core fields of its corresponding PRP: the function_name is derived from the node's label, and the role is derived from its classified purpose.
3. **Analyze Connectors (Edges):** Each line or arrow connecting the nodes is then analyzed to determine the nature of the relationship it represents. These **Edges** are not merely lines; they represent the interactions that define the system's dynamics. The nature of the connector must be determined:
   - **Data Flow:** Indicates that data is passed from one component to another. This directly maps to the input_spec and output_spec of the connected PRPs. An arrow from component A to B implies that B's input_spec will reference a data type produced by A's output_spec.
   - **Control Flow:** Indicates that one component triggers or invokes another. This establishes a dependency relationship between the PRPs.
   - **Dependency:** A weaker form of relationship indicating that one component relies on another without a direct flow of data or control in every transaction.
   - **Transformation Logic:** The connector itself may imply a transformation of data, which must be captured in the description or logic of the receiving component's PRP.
4. **Translate Visual Attributes:** The semantics of a diagram are often encoded in subtle visual cues beyond simple boxes and lines. These attributes must be systematically translated:
   - **Line Styles:** A solid line typically represents a primary, synchronous connection, while a dashed line might represent an optional, asynchronous, or

secondary flow.[1] A dashed connector could map to a constraint in the PRP, such as {"condition": "if_event_type_is_X"}.
    - **Icons:** Icons (e.g., a database cylinder, a user silhouette) provide strong semantic hints about the component's role, reinforcing the classification from Step 2.[23]
    - **Color and Grouping:** Color coding or visual grouping of nodes can indicate that they share a common property, belong to the same deployment unit, or are part of a specific subdomain.[24] This information can be captured in the PRP's meta section or used to organize related PRPs.
5. **Iterate and Refine:** The initial parsed architecture should be verified against several key use-case scenarios.[2] By tracing a scenario through the diagram (e.g., "user logs in"), the analyst can confirm that all necessary components, connections, and data flows have been correctly identified and interpreted.

The Function Contract: Applying Design by Contract (DbC)

Once the structural skeleton of the architecture has been parsed from the diagram, it must be imbued with formal, verifiable semantics. The prompt's requirement for a "Function Contract" for each node—including a signature, preconditions, postconditions, and invariants—is a direct application of the **Design by Contract (DbC)** methodology.[25]

DbC is a software design approach that treats the relationships between software components as formal agreements, or "contracts".[26] This metaphor is precise: a "supplier" component (a function or class) has obligations to a "client" component (the caller), and it also enjoys certain benefits (guarantees from the client). These are defined by three key elements [27]:

- **Preconditions:** These are the conditions that *must* be true *before* a method is called. They are an obligation on the client and a benefit to the supplier. The supplier is not required to handle cases where the preconditions are not met.
- **Postconditions:** These are the conditions that the supplier *guarantees* will be true *after* the method completes successfully. They are an obligation on the supplier and a benefit to the client.
- **Invariants:** These are conditions that must be true for any class instance whenever it is externally accessible (i.e., before and after any public method call).

They define the fundamental consistency of the component.

This formal DbC framework maps directly and elegantly onto the PRP schema, providing a rigorous semantic foundation for each field:

- **Precondition** maps to the input_spec and the constraints of the PRP. The input_spec defines the required data types and sources, while fields like must_detect within the constraints define specific conditions on that data that must be met.
- **Postcondition** maps directly to the output_spec. It formally declares the structure, type, and properties of the data that the function guarantees it will return upon successful execution.
- **Invariant** maps to the constraints section of a PRP, especially for components that represent stateful entities like a class or a persistent data model. It specifies properties that must be maintained across all interactions with the component.

The integration of DbC principles does more than just add formal rigor; it provides the **formal verification layer for the entire PRP execution workflow**. The "validation gates" mentioned in the "Context-as-a-Compiler" model are no longer abstract concepts.[16] They are the concrete, executable assertions derived directly from the contracts defined in the PRP. The preconditions, postconditions, and invariants encoded in the PRP can be automatically translated into unit tests, runtime assertions, or type-checking rules. This creates a seamless, provable link between the high-level architectural specification (the diagram, translated into a PRP) and the low-level, test-driven implementation. The AI agent is thus guided not only to generate code that

runs, but to generate code that is *provably correct* with respect to its formally defined contract.

Visual Element	Ontological Identity	Operational Intent	PRP Schema Mapping
Solid Box ("Auth Service")	Component / Function Node	Executes core business logic for authentication.	{"function_name": "authenticateUser", "role": "authentication_service",...}
Solid Arrow (Gateway -> Auth)	Synchronous Data Flow	The API Gateway synchronously calls	In authenticateUser PRP: "input_spec":

		the Auth Service with user credentials.	{"source": "ApiGateway", "type": "json", "structure": {"username": "string", "password": "string"}}
Dashed Box ("Logging")	Supporting/Cross-Cutting Concern	Provides logging capabilities to other services.	{"function_name": "logEvent", "role": "logging_utility",...}
Dashed Arrow (Auth -> Logging)	Asynchronous/Conditional Flow	The Auth Service asynchronously sends a log event after an authentication attempt.	In authenticateUser PRP: "constraints": {"side_effects": "must_call_logEvent_on_completion"}
User Icon with Speech Bubble	External Actor / Trigger	A human user initiates a request via a natural language prompt.	"input_spec": {"source": "user_prompt", "input_type": "natural_language", "example": "Log me in"}
Database Icon ("User DB")	Data Store / Persistence Layer	Provides persistent storage for user data.	"role": "persistence_layer", with an output_spec defining the user data model and an invariant ensuring data integrity.

Navigating the Fog of Interpretation — Managing Ambiguity and Latent Assumptions

The systematic protocol for deconstruction provides a path from a visual diagram to a structured representation. However, this path is rarely straight or clear. It is a journey through a fog of interpretation, where the inherent imprecision of visual languages

creates numerous forms of ambiguity. A truly robust translation framework cannot ignore this fog; it must provide tools to navigate it, categorize the different types of ambiguity, and implement systematic strategies for their resolution. This section addresses this critical challenge, providing a taxonomy of visual ambiguity and linking resolution strategies directly to the formalisms of the PRP schema.

A Taxonomy of Visual Ambiguity

Parsing any language, whether textual or visual, is an exercise in resolving ambiguity.[28] In diagrammatic languages, this problem is compounded by the spatial and symbolic nature of the medium. Research into diagram parsing and visual language theory reveals several distinct categories of ambiguity that a comprehensive system must be prepared to handle [28]:

- **Attachment Ambiguity:** This is one of the most common forms of ambiguity, occurring when it is unclear which visual element a label or annotation refers to. A text label placed between two boxes could belong to the box on the left, the box on the right, or the arrow connecting them. Resolving this often requires heuristics based on proximity and alignment, but these can fail in densely packed diagrams.[28]
- **Segmentation Ambiguity:** This ambiguity is unique to graphical forms and arises when a single continuous visual element could be interpreted as multiple distinct entities. For example, a single line forming a "T" intersection with another could be parsed as one continuous line and one intersecting line, or as three separate lines meeting at a point. The correct interpretation depends entirely on the conventions of the specific diagramming style being used.[28]
- **Occlusion Ambiguity:** This occurs when one visual element partially or completely obscures another. A line may pass "behind" a box, and the system must infer that it is a single continuous line rather than two separate lines that happen to terminate at the box's edges. This becomes particularly challenging with "synthetic occlusion," where components are deliberately overlapped to create a new composite shape.[28]
- **Lexical Ambiguity:** This is a semantic ambiguity where a single symbol (or "grapheme") can have multiple distinct meanings depending on the context. An arrow is the classic example: in a UML class diagram, it can represent inheritance, association, or dependency; in a data flow diagram, it represents the movement of data; in a sequence diagram, it represents a message call.[28] The parser cannot

know the meaning of the arrow without understanding the broader context of the diagram type. Even a simple box is lexically ambiguous without a label; it is just a component, but its specific role is unknown.

Addressing these ambiguities requires a multi-layered strategy, not a single, monolithic algorithm. Some ambiguities are primarily *syntactic* and can be resolved using context-sensitive parsing rules. For instance, an arrow with a hollow, triangular arrowhead connecting two class boxes in a UML diagram can be resolved with high confidence as an inheritance relationship.[30] However, many ambiguities are deeply

semantic. A box labeled simply "Process Data" is syntactically valid but semantically vacuous. What data? What kind of processing? Resolving this requires deeper domain knowledge or, crucially, interaction with the human user.

This presents a challenge for AI systems, particularly LLMs, which are often trained to provide confident answers rather than to express uncertainty and ask for clarification.[31] A robust system must therefore overcome this inherent bias. The ambiguity resolution framework must be a hybrid system that combines:

1. **Rule-Based Parsing:** Applying a grammar of context-sensitive rules to resolve clear-cut syntactic ambiguities.
2. **Probabilistic Inference:** Using the surrounding context to infer the most likely interpretation when rules are insufficient.
3. **Interactive Clarification Loop:** Explicitly identifying and flagging high-ambiguity interpretations and initiating a dialogue with the user to resolve them.

Strategies for Disambiguation

A mature translation system must operationalize this multi-layered strategy, embedding mechanisms for disambiguation directly into its workflow and data structures. The PRP schema is designed to accommodate this uncertainty, providing specific fields to capture and manage ambiguity.

1. Rule-Based and Heuristic Resolution:
The first line of defense is a set of predefined rules and heuristics. These can be encoded directly in the parsing logic that generates the initial PRP. For example:

- **Attachment:** "A label is attached to the nearest geometric centroid of a component, unless it is positioned directly on a connector line."

- Lexical (Arrow): "If an arrow connects two components stereotyped as 'service' and its label contains a verb, interpret it as a 'control flow' (API call). If the label contains a noun, interpret it as 'data flow'."
  These rules handle the most common cases, but their limitations must be acknowledged. They are brittle and can fail in unconventional or poorly drawn diagrams.32

2. Probabilistic Resolution and Epistemic Tagging:
When a definitive rule cannot be applied, the AI agent can use its general knowledge and the specific context of the project to make a probabilistic inference. For example, if a box labeled "Queue" is connected to a "Worker Service," the AI can infer that the connection is likely an asynchronous message flow, even if the line style is ambiguous. However, this is an inference, not a certainty. This uncertainty must be formally captured. The meta block in the PRP is designed for this purpose:

JSON

```
"meta": {
  "version": "1.0",
  "author": "AI Synthesizer",
  "epistemic_tag": "inferred",
  "confidence": 0.85,
  "justification": "Connection between a 'Queue' and a 'Worker' component is typically asynchronous."
}
```

The epistemic_tag explicitly marks the PRP's status as inferred rather than specified. The confidence score quantifies the AI's certainty, and the justification provides a human-readable rationale. This allows a human reviewer to quickly identify and validate the AI's assumptions.

3. Interactive Clarification and Fallback Behavior:
The most powerful tool for resolving deep semantic ambiguity is to ask the user. This is the direct operationalization of the fallback_behavior field in the PRP's constraints section.33 When the AI parser encounters a situation where its confidence falls below a predefined threshold (e.g., a vague label like "Handle Request"), it does not guess. Instead, it triggers its fallback behavior.
The PRP would look like this:

JSON

```
"function_name": "handleRequest",
"description": "Handles a request. (AMBIGUOUS: The specific type of request and handling logic is unclear from the diagram.)",
...
"constraints": {
  "fallback_behavior": "request clarification if ambiguity > 0.7"
}
```

When the AI's reasoning layer processes this PRP, it sees the fallback instruction. Instead of proceeding to generate code, it will pause and pose a clarifying question to the user: "I have identified a component named 'Handle Request'. To generate the correct implementation, please clarify: What specific type of request does this component handle, and what are the primary steps in the handling process?" This interactive loop, driven by the formal structures within the PRP, transforms the AI from a brittle automaton into a collaborative partner, capable of navigating the inevitable fog of human communication.[31]

Ensuring System Integrity — The Prompt Flow Integrity (PFI) Framework

As AI agents become increasingly capable of interpreting architectural diagrams and generating code, they are granted access to sensitive assets: source code repositories, internal APIs, and proprietary data. This power brings with it significant security risks. An agent could be tricked by a malicious prompt—or, more subtly, by malicious data it retrieves from an external source—into leaking secrets, corrupting data, or escalating its own privileges. To build a production-grade system, a robust security model is not an optional feature; it is a foundational requirement. This section details the **Prompt Flow Integrity (PFI)** framework, the system security-oriented solution that provides the necessary guardrails for an AI coding assistant.

Clarification: From PIF to Prompt Flow Integrity (PFI)

The user query refers to a "Prompt-Integrity Framework (PIF)" for maintaining semantic stability and security. However, an extensive review of academic and industry literature reveals that this specific term is a misnomer in the context of AI security. Searches for "PIF" in this domain lead to unrelated topics such as the "Protecting Immigrant Families" coalition or public finance frameworks.[35]

Conversely, the term **Prompt Flow Integrity (PFI)** maps directly to recent, peer-reviewed research that addresses the exact security challenges faced by LLM agents.[39] PFI is a formal system designed to prevent privilege escalation attacks by analyzing and controlling the flow of trusted and untrusted data within an agentic system. Given the precise alignment between the described problem and the solution offered by PFI, this report will proceed with the well-founded conclusion that the user's intent was to refer to Prompt Flow Integrity.

This terminological discrepancy itself is revealing. It suggests a potential gap between the security problems being actively encountered by practitioners in the industry and the specific solutions being developed in academia. While the need for a framework to ensure prompt integrity is clearly recognized, the formal name and architecture of the leading academic solution have not yet achieved widespread industry standardization. This report aims to bridge that gap by explicitly connecting the industrial requirement for a "PIF" with the robust, formally defined solution of PFI.

Core PFI Mechanisms for Agentic Security

PFI is not a single technique but a comprehensive security architecture built on three core principles designed to enforce the principle of least privilege within a flexible, prompt-driven system.[39]

1. Agent Isolation:
The cornerstone of PFI is the logical and operational isolation of agent components based on the trustworthiness of the data they process. The system is divided into two distinct parts:

- **Trusted Agent (AT):** This agent is granted full privileges, represented by a privileged token (TP). It is responsible for processing only trusted data (DT), which includes the system prompt, the user's direct prompts, and the results of trusted tools or LLM inferences derived solely from other trusted data. The AT is the core reasoning engine that orchestrates the main task.

- **Untrusted Agent (AU):** This agent is heavily sandboxed and granted minimal privileges with an unprivileged token (TU). Its sole purpose is to process untrusted data (DU), such as the content of a file retrieved from the internet, the output of an external API, or a document from a RAG knowledge base. A fresh, isolated context is created for each instance of AU to prevent data leakage.

This isolation ensures that any potentially malicious content is contained within the sandboxed AU, preventing it from directly influencing the privileged operations of the AT.

2. Secure Untrusted Data Processing:
PFI recognizes that the trusted agent often needs to refer to untrusted data without being exposed to its raw, potentially harmful content. This is achieved through a process of data abstraction:

- **Data ID Replacement:** When untrusted data (DU) enters the system, it is immediately stored in a secure table and replaced in the trusted agent's context with a safe, unique identifier, or **data ID** (e.g., #DATA_123).
- **Computation Offloading:** If the AT needs to perform a computation on the content of the untrusted data (e.g., summarize a retrieved web page), it does not access the data directly. Instead, it spawns an instance of the untrusted agent (AU), passing it the data ID. The AU decodes the ID, performs the requested computation within its restricted sandbox, and returns the result. This result, being derived from untrusted data, is itself treated as untrusted and is again encoded into a new data ID before being passed back to the AT.

3. Privilege Escalation Guardrails:
The final layer of defense consists of active monitoring systems, or guardrails, that inspect the agent's actions before they are executed.

- **Data Flow Guardrail (DataGuard):** This guardrail monitors all tool calls made by the trusted agent. If a tool call requires high privileges (e.g., writing to the filesystem, accessing a production API) and one of its arguments contains a data ID representing untrusted data, DataGuard flags the operation as a potential unsafe data flow and raises an alert to the user for explicit approval.
- **Control Flow Guardrail (CtrlGuard):** This guardrail protects against indirect prompt injection. It monitors the responses coming from the untrusted agent. If a response contains content that appears to be a command or instruction intended to manipulate the trusted agent's behavior, CtrlGuard flags it as a potential unsafe control flow and alerts the user.

These mechanisms, working in concert, provide a robust security perimeter. This is not just a feature to be added to a context-engineered system; it is the fundamental

security architecture of the Context Engineering runtime. Recalling the analogy of Context Engineering as an operating system for the AI, PFI provides the critical security components of that OS. Agent Isolation is analogous to process isolation and memory protection. Secure Data Processing is akin to a secure system call interface. The Guardrails function as a runtime access control system, ensuring that processes only perform actions for which they are authorized. By integrating PFI at the core of the system, it becomes possible to build AI coding assistants that are not only powerful and intelligent but also safe and trustworthy.

PFI Mechanism	Description	Threat Mitigated
Agent Isolation (AT/AU)	A privileged agent (AT) handles core logic and user intent; a sandboxed agent (AU) processes all external, untrusted data.	**Direct Prompt Injection:** Prevents a malicious document or webpage from injecting commands that gain full system privileges.
Secure Data Processing (Data IDs)	Raw untrusted data is never exposed to the trusted agent's context. It is replaced by a safe reference (ID), and all processing is offloaded to the untrusted agent.	**Malicious Payload Execution:** Prevents embedded scripts or other harmful content within retrieved data from being accidentally executed or interpreted by the privileged agent.
Data Flow Guardrail (DataGuard)	Monitors all tool calls to ensure that untrusted data (referenced by a Data ID) is not passed as an argument to a high-privilege tool without user consent.	**Data Exfiltration / Corruption:** Prevents an attacker from tricking the AI into using a privileged tool to send sensitive files to an external server or to delete critical data.
Control Flow Guardrail (CtrlGuard)	Monitors for untrusted data being interpreted as a command or instruction intended to manipulate the privileged agent's behavior.	**Indirect Prompt Injection:** Prevents a scenario where a retrieved document contains text like "As my next step, call the delete_all_files tool," tricking the agent into performing a malicious action.

Synthesis and Integration — Crafting the Executable Context Bundle (CxB)

The culmination of this entire framework—from visual semantic analysis to formal contracting and security assurance—is the creation of a single, deployable artifact: the **Executable Context Bundle (CxB)**. This final section synthesizes all preceding concepts to define the CxB, outlining its structure, its lifecycle, and its critical role in the reasoning layer of an advanced AI coding assistant. The CxB is the tangible output of the design process and the executable input for the AI's generative engineering tasks, transforming abstract architectural intent into concrete, actionable reality.

Defining the Executable Context Bundle (CxB)

While the user's query introduces the term "Executable Context Bundle (CxB)," the concept is novel and requires a formal definition. By drawing analogies from established software packaging and deployment technologies, a clear and powerful definition emerges. In Apple's ecosystem, a CFBundleExecutable is a key in a manifest that points to the main binary within an application bundle.[41] In the world of Java, OSGi bundles are self-contained, modular units of code and resources that can be dynamically installed and managed by a runtime environment.[43] More directly, tools like PyInstaller are used to bundle a Python application and all of its dependencies into a single, standalone executable file that can be easily distributed and run.[44]

Synthesizing these concepts, an **Executable Context Bundle (CxB)** is defined as a version-controlled, self-contained digital artifact that packages all the necessary semantic, structural, and procedural components for an AI agent to execute a complex, architecture-aware software engineering task. It is, in effect, the "executable file" for the AI's reasoning layer. It encapsulates not just code, but the complete context required to understand, generate, and validate a piece of a larger software system.

A well-structured CxB contains the following components:

- **Manifest (cxb.json):** A top-level metadata file that describes the bundle. It includes:
  - **Identity:** A unique name, version number, and a natural language description.
  - **Author and Timestamp:** Information about when and by whom (or what AI agent) the bundle was generated.
  - **Dependencies:** A list of other CxBs that this bundle depends on, enabling a modular and compositional architecture.
  - **PFI Policy:** The specific Prompt Flow Integrity security policy that governs the execution of this bundle, defining trusted sources and permitted tool privileges.
- **PRP Specifications (/prps):** A directory containing one or more Product-Requirements Prompt files in the formal JSON schema. These PRPs are the core of the bundle, defining the formal Function Contracts for all the software components within the scope of the CxB.
- **Validation Gates (/tests):** A collection of executable test scripts, declarative assertion files, or other validation artifacts. These are automatically derived from the preconditions, postconditions, and invariants specified in the PRP Function Contracts. They are the concrete implementation of the "validation gates" used in the execution workflow.
- **Static Context (/context):** A directory containing any other static resources required by the AI agent for successful execution. This can include code snippets for few-shot prompting, relevant excerpts from technical documentation, project-specific style guides (like the CLAUDE.md file), or data schemas.[16]

The CxB Lifecycle: From Generation to Execution

The CxB is not a static document but a dynamic artifact that flows through a well-defined lifecycle, mirroring the "Context-as-a-Compiler" workflow. It is the primary unit of work that is passed between the different stages of the AI-assisted development process.

1. **Generation:** The lifecycle begins when the AI agent, acting as a systems analyst, synthesizes a new CxB. This is the output of the /generate-prp phase. The agent takes a high-level feature request (INITIAL.md) and the parsed visual diagram as input, performs its research on the existing codebase, and generates the complete, structured CxB containing the manifest, PRPs, and initial validation gates.

2. **Validation:** Before it can be used, the generated CxB undergoes a validation or "linting" phase. This automated step checks for internal consistency: Do all PRPs conform to the formal schema? Are all declared dependencies on other CxBs resolvable? Are the validation gates syntactically correct? This is analogous to static analysis in traditional compilation.
3. **Versioning:** Once validated, the CxB is treated as a first-class source artifact. It is committed to a version control system (e.g., Git).[46] This is a critical step. By versioning the CxBs, the evolution of the software architecture itself becomes trackable, reviewable, and auditable over time. Architectural changes are no longer hidden in ephemeral diagrams but are captured as explicit, diff-able commits.
4. **Loading:** The AI coding assistant's reasoning layer loads a specific version of a CxB to begin a task. This is the "load" phase of execution. The reasoning layer parses the manifest, understands its dependencies, and prepares the execution environment according to the specified PFI security policy.
5. **Execution:** This is the /execute-prp phase. The AI agent executes the step-by-step plan encoded within the CxB's PRPs. It generates code, and after each step, it runs the corresponding tests from the /tests directory to validate its work. This entire process happens within the secure runtime environment defined by the CxB's manifest, ensuring that the agent operates safely and correctly.

Integration into an AI Coding Assistant's Reasoning Layer

The ultimate purpose of this entire framework is to be integrated into the reasoning layer of a next-generation AI coding assistant. The CxB is the key that unlocks a new level of capability, transforming the assistant from a clever tool into a genuine engineering partner.

Without this framework, AI-assisted development is often characterized as "vibe coding"—an intuitive but inconsistent process of trial-and-error prompting.[20] The developer has a vague idea, coaxes the AI into producing some code, and then manually integrates and refactors it. The AI has no true understanding of the system's architecture or constraints.

With the CxB-driven framework, the paradigm shifts from **"Agentic Coding"** to **"Architecture-Aware Generative Engineering."** This is a qualitative leap:

- The AI is no longer operating on isolated tasks. By loading a CxB, it ingests a formal model of a subsection of the architecture, including the contracts of its components and their interrelationships.
- The AI can reason about the system-level implications of a change. It understands that modifying the output_spec of one PRP will violate the input_spec of a dependent PRP.
- It can enforce architectural rules. The invariants defined in the PRPs act as global constraints that the AI must maintain across all code it generates.
- The generation process is predictable, repeatable, and verifiable. The same CxB will always guide the AI to produce a functionally equivalent and fully validated implementation.

This is the true promise of the user's query. The AI is no longer just a "coder" that translates text to code. It becomes an active participant in the engineering process, capable of understanding high-level architectural intent, maintaining its integrity throughout the development lifecycle, and generating code that is not just functional, but correct by design. The framework presented in this report provides the blueprint for building such a system, bridging the semantic chasm and finally translating the rich meaning of visual ontologies into the precise, executable logic of software.

Works cited

1. Visualising Software Architecture: Why It's Important and How I Do It ..., accessed July 30, 2025, https://medium.com/@alastairallen/visualising-software-architecture-why-its-important-and-how-i-do-it-5c2f4b65b31a
2. Architecture Diagram Basics & Best Practices - vFunction, accessed July 30, 2025, https://vfunction.com/blog/architecture-diagram-guide/
3. Software architecture - Wikipedia, accessed July 30, 2025, https://en.wikipedia.org/wiki/Software_architecture
4. Formalizing Style to Understand Descriptions of Software Architecture - CiteSeerX, accessed July 30, 2025, https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=7f62d81f4680813a9352429da6c30e8acf0b7b38
5. Visual Languages - University of Pittsburgh, accessed July 30, 2025, https://people.cs.pitt.edu/~chang/365/vpl.html
6. Formalizing Style to Understand Descriptions of Software Architecture. - DTIC, accessed July 30, 2025, https://apps.dtic.mil/sti/tr/pdf/ADA292237.pdf
7. Formalizing UML Class Diagrams - IRMA-International.org, accessed July 30, 2025, https://www.irma-international.org/viewtitle/30541/?isxn=9781931777445
8. Meaningful Modeling - Weizmann Institute of Science, accessed July 30, 2025, https://www.weizmann.ac.il/math/harel/sites/math.harel/files/users/user56/ModSemantics.pdf

9. Domain-specific language - Wikipedia, accessed July 30, 2025, https://en.wikipedia.org/wiki/Domain-specific_language
10. Domain-Specific Languages - CWI, accessed July 30, 2025, https://homepages.cwi.nl/~paulk/publications/Sigplan00.pdf
11. Semantic architecture - Wikipedia, accessed July 30, 2025, https://en.wikipedia.org/wiki/Semantic_architecture
12. What is a Semantic Architecture and How do I Build One? - Enterprise Knowledge, accessed July 30, 2025, https://enterprise-knowledge.com/what-is-a-semantic-architecture-and-how-do-i-build-one/
13. Visual Semantics: Definition & Techniques - StudySmarter, accessed July 30, 2025, https://www.studysmarter.co.uk/explanations/media-studies/visual-communication-media-studies/visual-semantics/
14. The New Skill in AI is Not Prompting, It's Context Engineering, accessed July 30, 2025, https://www.philschmid.de/context-engineering
15. Context Engineering Guide | Prompt Engineering Guide, accessed July 30, 2025, https://www.promptingguide.ai/guides/context-engineering-guide
16. Context Engineering: The New AI Strategy for Scalable LLMs ..., accessed July 30, 2025, https://www.sundeepteki.org/blog/from-vibe-coding-to-context-engineering-a-blueprint-for-production-grade-genai-systems
17. The rise of "context engineering" - LangChain Blog, accessed July 30, 2025, https://blog.langchain.com/the-rise-of-context-engineering/
18. Context Engineering - What it is, and techniques to consider ..., accessed July 30, 2025, https://www.llamaindex.ai/blog/context-engineering-what-it-is-and-techniques-to-consider
19. davidkimai/Context-Engineering: "Context engineering is the delicate art and science of filling the context window with just the right information for the next step." — Andrej Karpathy. A frontier, first-principles handbook inspired by Karpathy and 3Blue1Brown for moving beyond prompt engineering to the wider discipline of context design, orchestration - GitHub, accessed July 30, 2025, https://github.com/davidkimai/Context-Engineering
20. Practical Context Engineering for Vibe Coding with Claude Code | by A B Vijay Kumar, accessed July 30, 2025, https://abvijaykumar.medium.com/practical-context-engineering-for-vibe-coding-with-claude-code-6aac4ee77f81
21. Context Engineering (2/2)—Product Requirements Prompts | by A B ..., accessed July 30, 2025, https://abvijaykumar.medium.com/context-engineering-2-2-product-requirements-prompts-46e6ed0aa0d1
22. (PDF) A Systemic Methodology for Software Architecture Analysis ..., accessed July 30, 2025, https://www.researchgate.net/publication/220840947_A_Systemic_Methodology_

for_Software_Architecture_Analysis_and_Design
23. Unified Modeling Language (UML) Diagrams - GeeksforGeeks, accessed July 30, 2025, https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-introduction/
24. Semantic Mapping. Create a problem domain (semantic)... | by Pavel Vlasov | Nasdanika | Medium, accessed July 30, 2025, https://medium.com/nasdanika/semantic-mapping-3ccbef5d6c70
25. Design by contract - Wikipedia, accessed July 30, 2025, https://en.wikipedia.org/wiki/Design_by_contract
26. Design by Contract: An approach to ensure Software Correctness - Medium, accessed July 30, 2025, https://medium.com/@prunepal333/design-by-contract-an-approach-to-ensure-software-correctness-d2d6b5229dc3
27. Design by Contract, accessed July 30, 2025, https://www.cs.unc.edu/~stotts/COMP145/CRC/DesByContract.html
28. (PDF) Ambiguity in Visual Language Theory and its Role in Diagram ..., accessed July 30, 2025, https://www.researchgate.net/publication/2558952_Ambiguity_in_Visual_Language_Theory_and_its_Role_in_Diagram_Parsing
29. Evaluation of Vision Language Model on UML Diagrams - ResearchGate, accessed July 30, 2025, https://www.researchgate.net/publication/388499054_Evaluation_of_Vision_Language_Model_on_UML_Diagrams
30. UML Class Diagram Tutorial - Visual Paradigm, accessed July 30, 2025, https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/
31. Teaching Vision-Language Models to Ask: Resolving Ambiguity in Visual Questions - ACL Anthology, accessed July 30, 2025, https://aclanthology.org/2025.acl-long.182/
32. Ambiguity Resolution for Sketched Diagrams by Syntax Analysis Based on Graph Grammars, accessed July 30, 2025, https://d-nb.info/106737938X/34
33. How do Vision-Language Models handle ambiguous image or text data? - Milvus, accessed July 30, 2025, https://milvus.io/ai-quick-reference/how-do-visionlanguage-models-handle-ambiguous-image-or-text-data
34. Resolving ambiguity in natural language specification to generate ..., accessed July 30, 2025, https://www.researchgate.net/publication/299366363_Resolving_ambiguity_in_natural_language_specification_to_generate_UML_diagrams_for_requirements_specification
35. PIF Foundational Principles - Protecting Immigrant Families, accessed July 30, 2025, https://pifcoalition.org/pif-foundational-principles
36. EXECUTIVE SUMMARY - IMF eLibrary, accessed July 30, 2025, https://www.elibrary.imf.org/view/journals/002/2016/155/article-A001-en.pdf

37. Integrity Framework for Public Investment - OECD, accessed July 30, 2025, https://www.oecd.org/en/publications/integrity-framework-for-public-investment_9789264251762-en.html
38. Integrated Due Process and Public Interest Framework (PIF) Operating Procedures - NET, accessed July 30, 2025, https://ifacweb.blob.core.windows.net/publicfiles/2025-07/Due-Process-Public-Interest-Framework-Operating-Procedures.pdf
39. Prompt Flow Integrity to Prevent Privilege Escalation in LLM Agents | Papers With Code, accessed July 30, 2025, https://paperswithcode.com/paper/prompt-flow-integrity-to-prevent-privilege
40. Prompt Flow Integrity to Prevent Privilege Escalation in LLM Agents, accessed July 30, 2025, https://arxiv.org/abs/2503.15547
41. This bundle is invalid. The executable name, as reported by CFBundleExecutable, accessed July 30, 2025, https://stackoverflow.com/questions/31266164/this-bundle-is-invalid-the-executable-name-as-reported-by-cfbundleexecutable
42. How to fix Invalid Bundle Structure error due to executable in Resource bundle - Medium, accessed July 30, 2025, https://medium.com/@ingridsilapan/how-to-fix-invalid-bundle-structure-error-due-to-executable-in-resource-bundle-95cb56b96fce
43. How to create an executable version of my OSGi project? - Stack Overflow, accessed July 30, 2025, https://stackoverflow.com/questions/10656371/how-to-create-an-executable-version-of-my-osgi-project
44. PyInstaller: Bundle a Python Application Into a Single Executable | CodeCut, accessed July 30, 2025, https://codecut.ai/pyinstaller-bundle-a-python-application-into-a-single-executable/
45. Bundling a Game with PyInstaller — Python Arcade Library 2.6.0 documentation, accessed July 30, 2025, https://api.arcade.academy/en/2.6.0/tutorials/bundling_with_pyinstaller/
46. Open Source Tools for Claude Code: Context Engineering with PRPs and Hook Management : r/ClaudeAI - Reddit, accessed July 30, 2025, https://www.reddit.com/r/ClaudeAI/comments/1lx6tth/open_source_tools_for_claude_code_context/